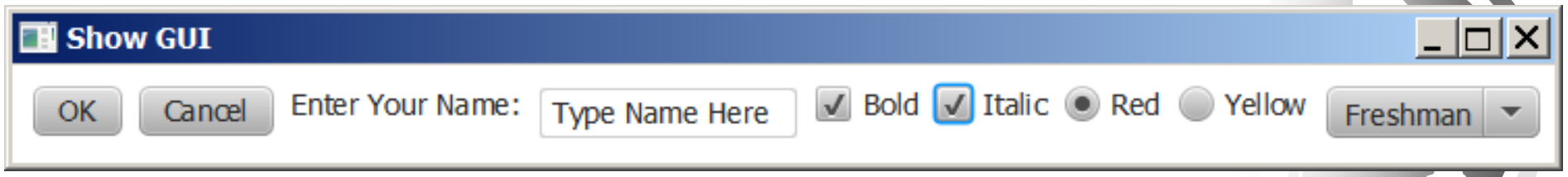
# Chapter 9 Objects and Classes



# Motivations

After learning the preceding chapters, you are capable of solving many programming problems using selections, loops, methods, and arrays. However, these Java features are not sufficient for developing graphical user interfaces and large scale software systems. Suppose you want to develop a graphical user interface (GUI) as shown below. How do you program it?



# Objectives

- To describe objects and classes, and use classes to model objects (§9.2).
- To use UML graphical notation to describe classes and objects (§9.2).
- To demonstrate how to define classes and create objects (§9.3).
- To create objects using constructors (§9.4).
- To access objects via object reference variables (§9.5).
- To define a reference variable using a reference type (§9.5.1).
- To access an object's data and methods using the object member access operator (**.**) (§9.5.2).
- To define data fields of reference types and assign default values for an object's data fields (§9.5.3).
- To distinguish between object reference variables and primitive data type variables (§9.5.4).
- To use the Java library classes **Date**, **Random**, and **Point2D** (§9.6).
- To distinguish between instance and static variables and methods (§9.7).
- To define private data fields with appropriate **get** and **set** methods (§9.8).
- To encapsulate data fields to make classes easy to maintain (§9.9).
- To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§9.10).
- To store and process objects in arrays (§9.11).
- To create immutable objects from immutable classes to protect the contents of objects (§9.12).
- To determine the scope of variables in the context of a class (§9.13).
- To use the keyword **this** to refer to the calling object itself (§9.14).



# OO Programming Concepts

- Object-oriented programming enables you to develop large-scale software and GUIs effectively.
- Object-oriented programming (OOP) involves programming using objects.
- An *object* represents an entity in the real world that can be distinctly identified.
- For example, a student, a desk, a circle, a button, and even a loan can all be viewed as objects.



# OO Programming Concepts

- An object has a unique identity, state, and behaviors.
- The *state* of an object consists of a set of *data fields* (also known as *properties*) with their current values.
- The *behavior* of an object is defined by a set of methods.



# Classes and Objects

- Objects of the same type are defined using a common class.
- A *class* is a template, blueprint or *contract* that defines what an object's data fields and methods will be.
- An object is an instance of a class.
- You can create many instances of a class.

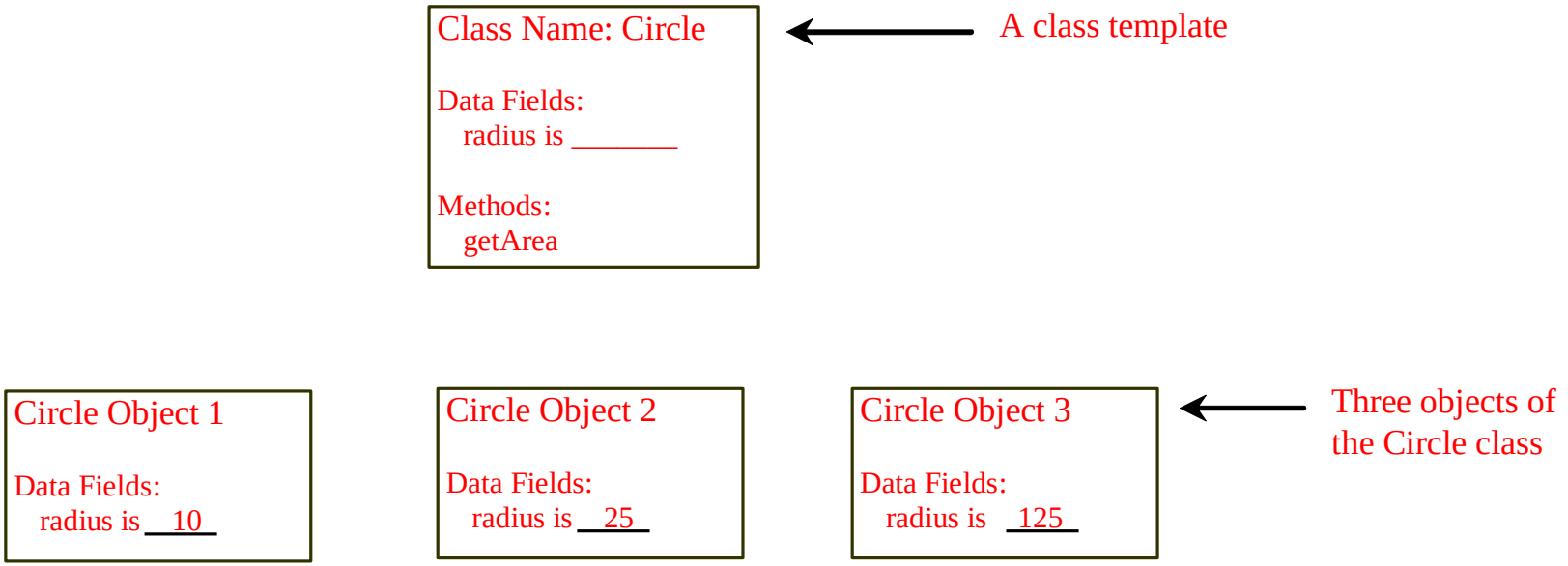


# Classes and Objects

- Creating an instance is referred to as *instantiation*.
- The terms *object* and *instance* are often interchangeable.
- The relationship between classes and objects is analogous to that between an apple-pie recipe and apple pies: You can make as many apple pies as you want from a single recipe.



# Objects




An object has both a state and behavior. The state defines the object, and the behavior defines what the object does.

# Classes

- *Classes* are constructs that define objects of the same type.
- A Java class uses variables to define data fields and methods to define behaviors.
- Additionally, a class provides a special type of methods, known as constructors, which are invoked to construct objects from the class.



# Classes

```
class Circle {
  /** The radius of this circle */
  double radius = 1.0;

  /** Construct a circle object */
  Circle() {
  }

  /** Construct a circle object */
  Circle(double newRadius) {
    radius = newRadius;
  }

  /** Return the area of this circle */
  double getArea() {
    return radius * radius * 3.14159;
  }
}
```

Data field

Constructors

Method



# Classes

- The **Circle** class is different from all of the other classes you have seen thus far.
- It does not have a **main** method and therefore cannot be run; it is merely a definition for circle objects.
- The class that contains the **main** method will be referred to in this book, for convenience, as the *main class*.



# UML Class Diagram

- The illustration of class templates and objects can be standardized using *Unified Modeling Language (UML)* notation.

- This notation is called a *UML class diagram*, or simply a *class diagram*.



# UML Class Diagram

- In the class diagram, the data field is denoted as

  dataFieldName: dataFieldType

- The constructor is denoted as

  ClassName(parameterName: parameterType)

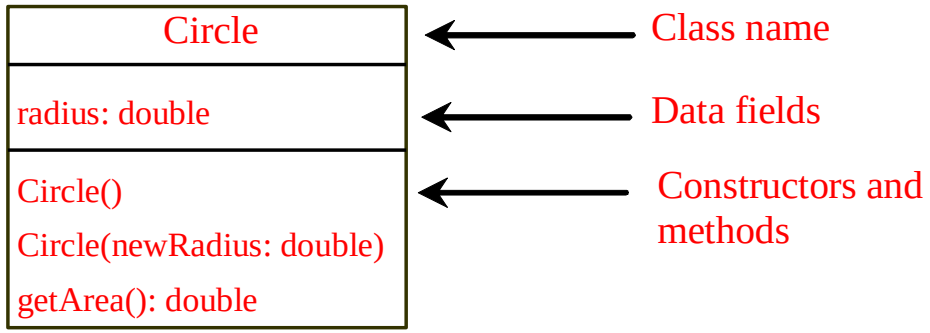
- The method is denoted as

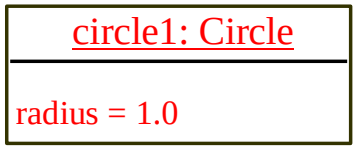
  methodName(parameterName: parameterType): returnType

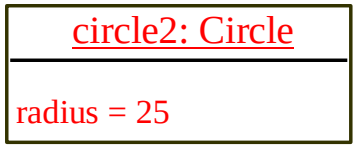


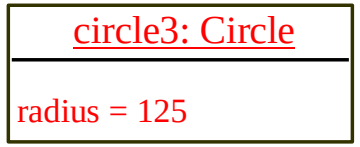
# UML Class Diagram

UML Class Diagram






# Example: Defining Classes and Creating Objects

Objective: Demonstrate creating objects, accessing data, and using methods.



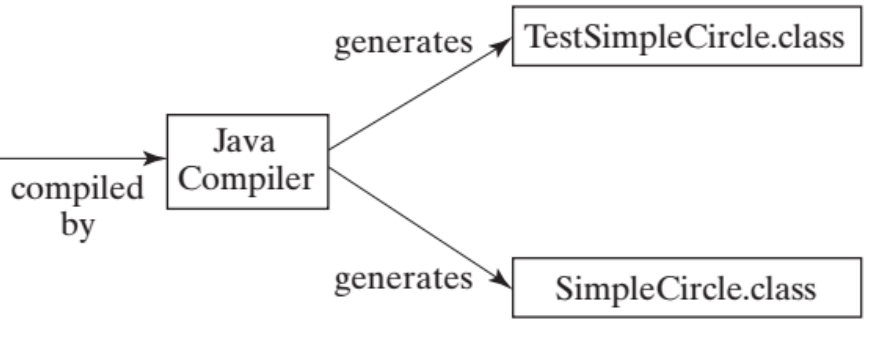
# Example: TestSimpleCircle.java

- The program contains two classes.
  - TestSimpleCircle
  - SimpleCircle
- You can put the two classes into one file, but only one class in the file can be a *public class*.
- The public class must have the same name as the file name.
- Each class in the source code is compiled into a **.class** file.

```
// File TestSimpleCircle.java

public class TestSimpleCircle {
  …
}

class SimpleCircle {
  …
}
```



# SimpleCircle.java

```
1 public class SimpleCircle {
2   /** Main method */
3   public static void main(String[] args) {
4     // Create a circle with radius 1
5     SimpleCircle circle1 = new SimpleCircle();
6     System.out.println("The area of the circle of radius "    + circle1.radius + " is " +
7      circle1.getArea());
8   }
9
10   double radius;
11
12   /** Construct a circle with radius 1 */
13   SimpleCircle() {
14       radius = 1;
15   }
16
17    /** Construct a circle with a specified radius */
18   SimpleCircle(double newRadius) {
19        radius = newRadius;
20    }
21
22   /** Return the area of this circle */
23   double getArea() {
24       return radius * radius * Math.PI;
25   }
```

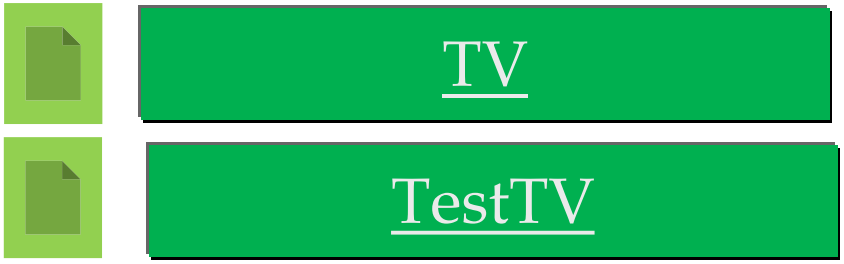


# Example: Defining Classes and Creating Objects

| TV | |
|---|---|
| `channel: int` | The current channel (1 to 120) of this TV. |
| `volumeLevel: int` | The current volume level (1 to 7) of this TV. |
| `on: boolean` | Indicates whether this TV is on/off. |
| `+TV()` | Constructs a default TV object. |
| `+turnOn(): void` | Turns on this TV. |
| `+turnOff(): void` | Turns off this TV. |
| `+setChannel(newChannel: int): void` | Sets a new channel for this TV. |
| `+setVolume(newVolumeLevel: int): void` | Sets a new volume level for this TV. |
| `+channelUp(): void` | Increases the channel number by 1. |
| `+channelDown(): void` | Decreases the channel number by 1. |
| `+volumeUp(): void` | Increases the volume level by 1. |
| `+volumeDown(): void` | Decreases the volume level by 1. |

The + sign indicates a public modifier.

The constructor and methods in the **TV** class are defined **public** so they can be accessed from other classes.






# Constructors

```
Circle() {
}
```

Constructors are a special kind of methods that are invoked to construct objects.

```
Circle(double newRadius) {
  radius = newRadius;
}
```



# Constructors, cont.

- A constructor with no parameters is referred to as a *no-arg constructor*.
- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even *void*.
- Constructors are invoked using the **new** operator when an object is created. Constructors play the role of initializing objects.



# Creating Objects Using Constructors

```
new ClassName();
```

Example:

```
new Circle();
```

```
new Circle(5.0);
```



# Default Constructor

- A class may be defined without constructors.
- In this case, a no-arg constructor with an empty body is implicitly defined in the class.
- This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.



# Declaring Object Reference Variables

To reference an object, assign the object to a reference variable.

To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

Example:

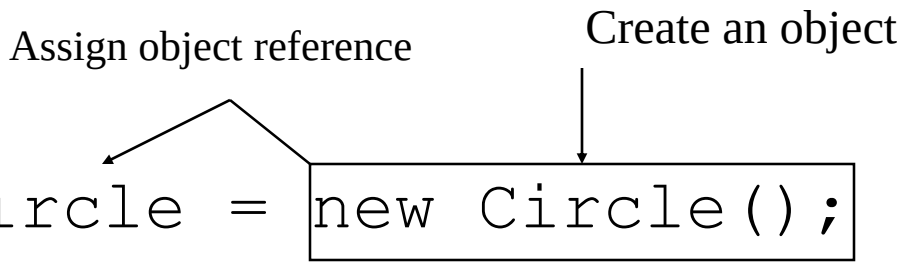
```
Circle myCircle;
```



# Declaring/Creating Objects in a Single Step

```
ClassName objectRefVar = new ClassName();
```

Example:



# Accessing Object's Members

- Referencing the object's data:

  `objectRefVar.data`

  *e.g.,* `myCircle.radius`

- Invoking the object's method:

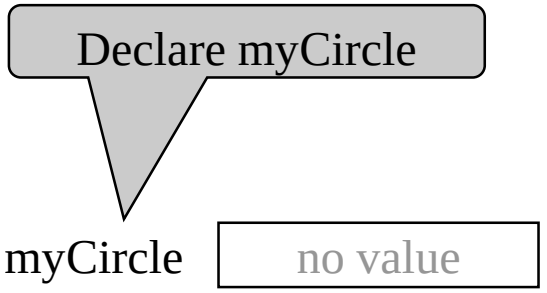
  `objectRefVar.methodName(arguments)`

  *e.g.,* `myCircle.getArea()`



animation

# Trace Code




**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



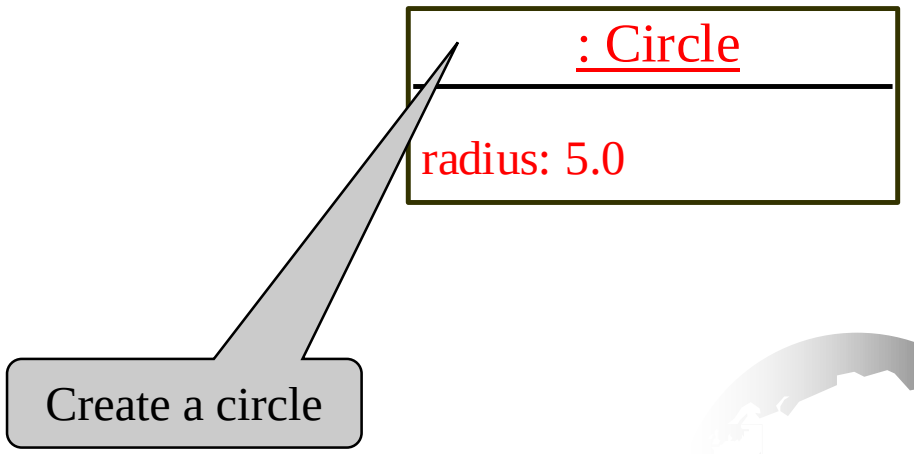
animation

# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



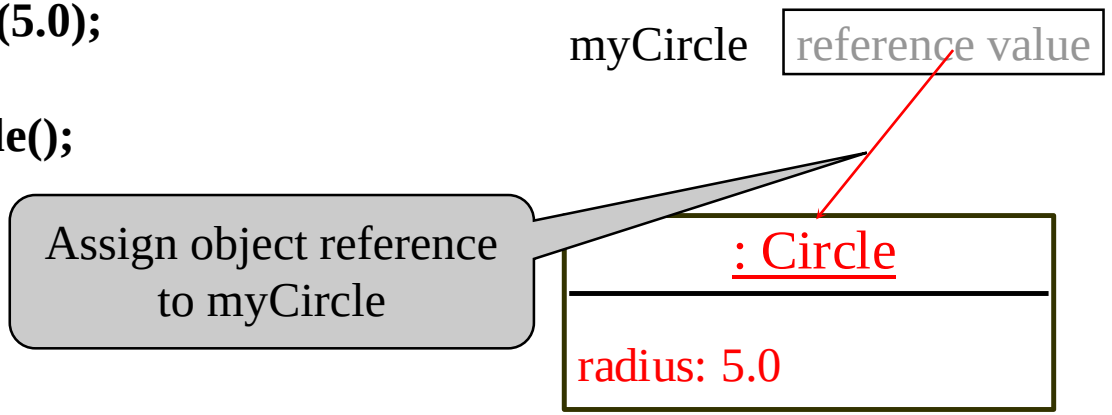
animation

# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



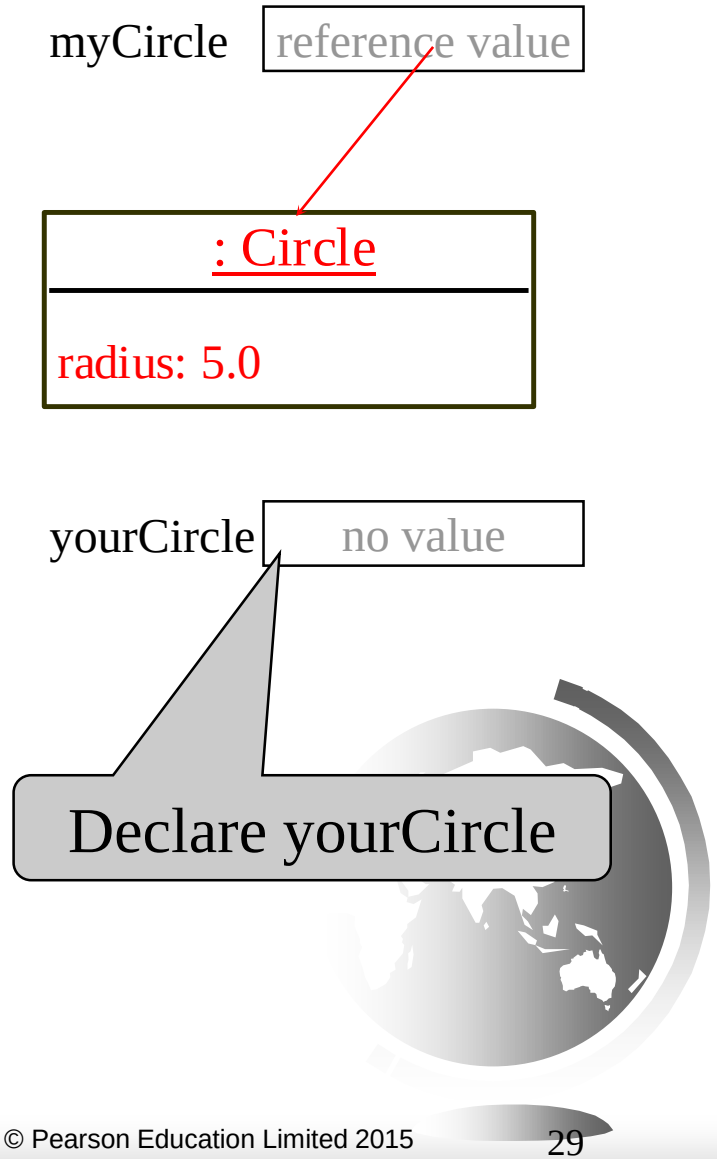
animation

# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



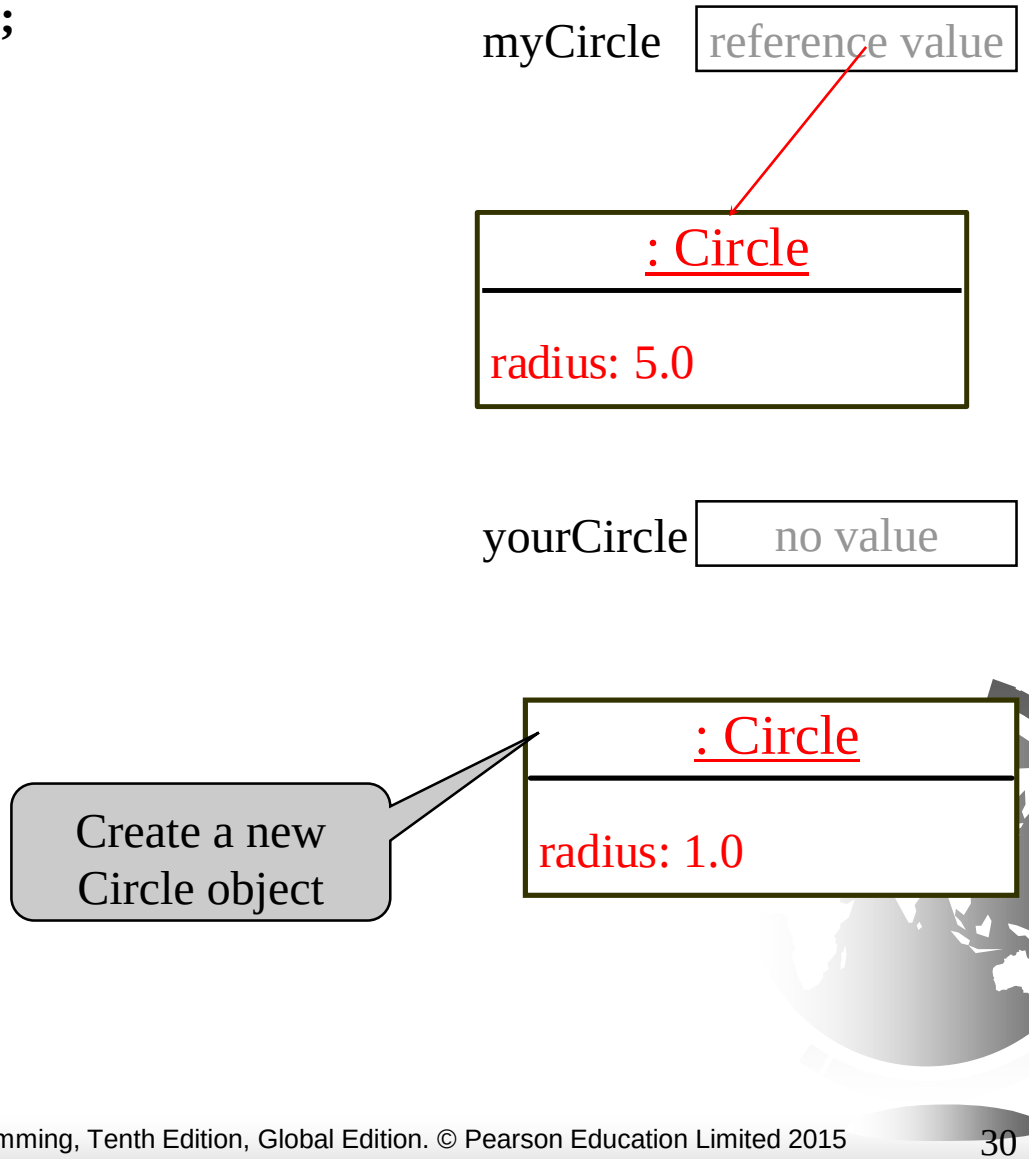
animation

# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



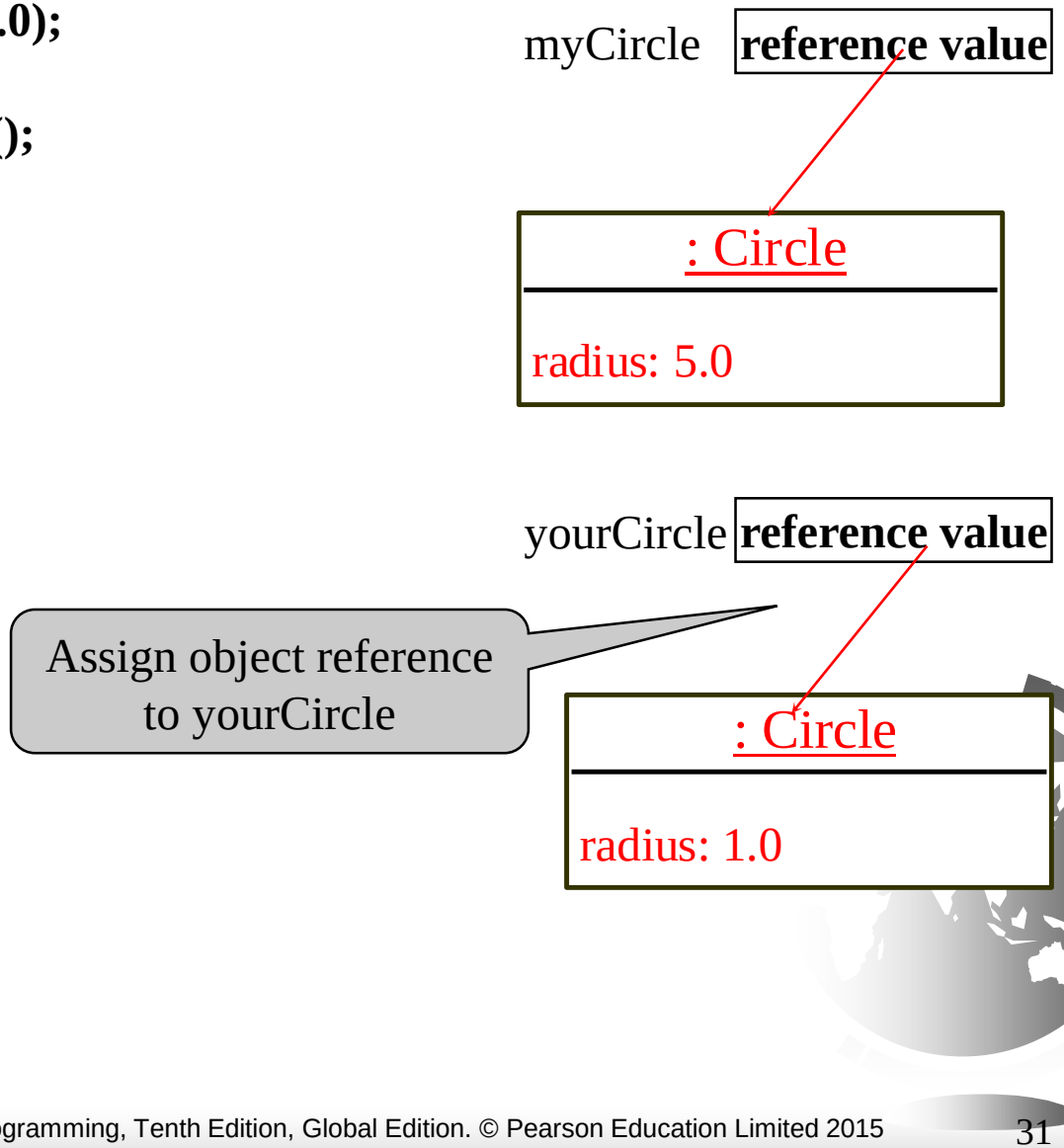
animation

# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**

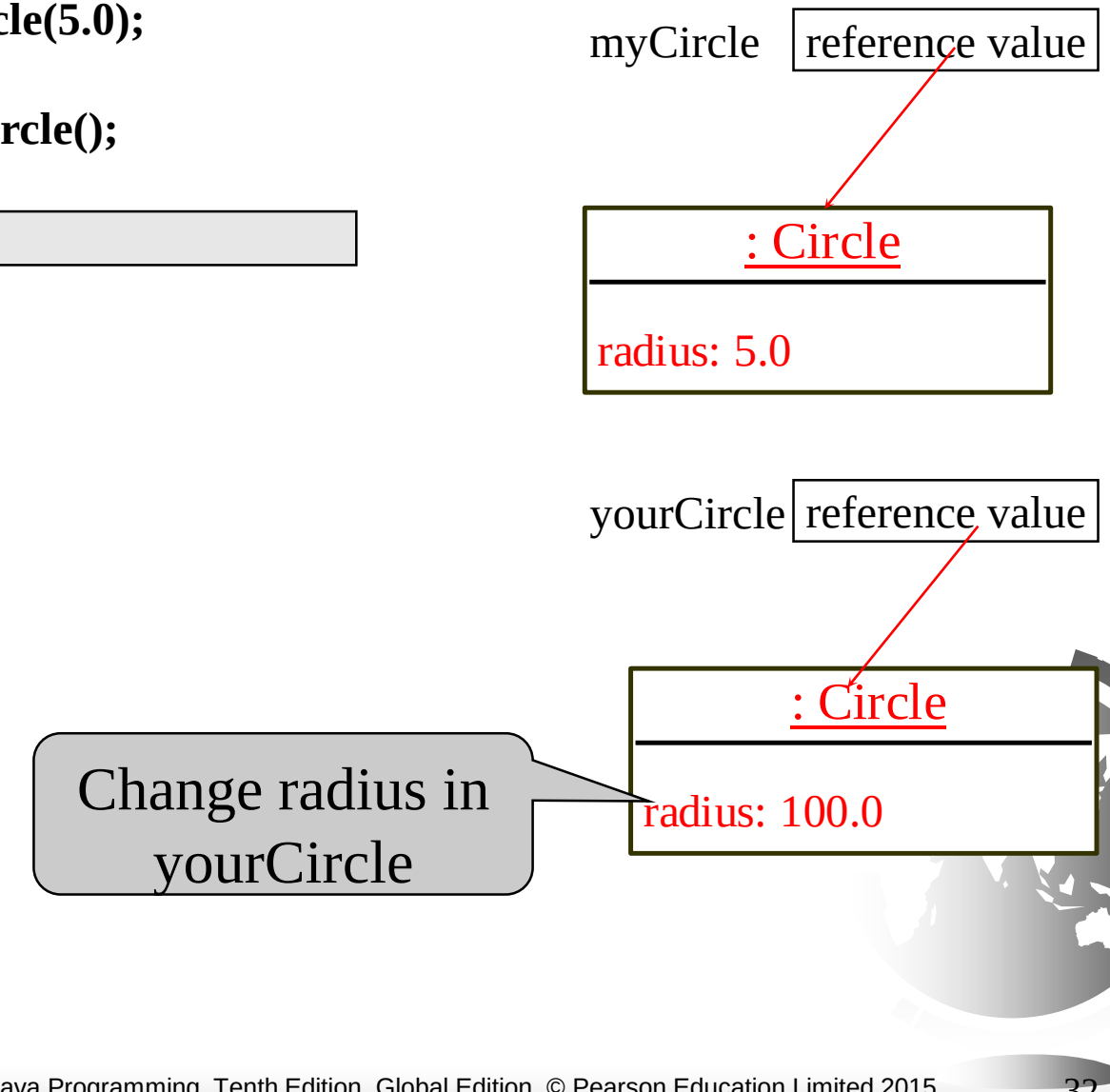


# Trace Code, cont.

**Circle myCircle = new Circle(5.0);**

**Circle yourCircle = new Circle();**

**yourCircle.radius = 100;**



# Caution

Recall that you use

Math.methodName(arguments) (e.g., Math.pow(3, 2.5))

to invoke a method in the Math class. Can you invoke getArea() using SimpleCircle.getArea()? The answer is no. All the methods used before this chapter are **static** methods, which are defined using the **static** keyword. However, getArea() is **non-static**. It must be invoked from an object using

objectRefVar.methodName(arguments) (e.g., myCircle.getArea()).

More explanations will be given in the section on "Static Variables, Constants, and Methods."



# Reference Data Fields

The data fields can be of reference types. For example, the following Student class contains a data field name of the String type.

```
public class Student {
  String name; // name has default value null
  int age; // age has default value 0
  boolean isScienceMajor; // isScienceMajor has default value false
  char gender; // c has default value '\u0000'
}
```



# The null Value

- If a data field of a reference type does not reference any object, the data field holds a special literal value, **null**.
- **null** is a literal just like **true** and **false**.
  - While **true** and **false** are Boolean literals, **null** is a literal for a reference type.



# Default Value for a Data Field

The default value of a data field is null for a reference type, 0 for a numeric type, false for a boolean type, and '\u0000' for a char type. However, Java assigns no default value to a local variable inside a method.

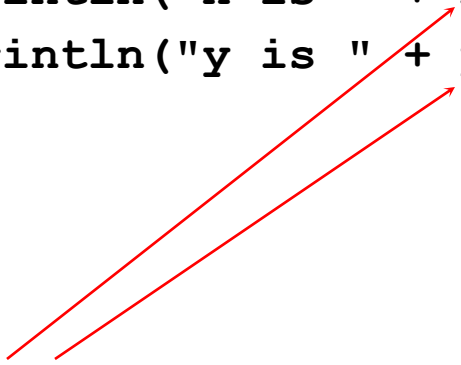
```
public class Test {
  public static void main(String[] args) {
    Student student = new Student();
    System.out.println("name? " + student.name);
    System.out.println("age? " + student.age);
    System.out.println("isScienceMajor? " + student.isScienceMajor);
    System.out.println("gender? " + student.gender);
  }
}
```



# Example

Java assigns no default value to a local variable inside a method.

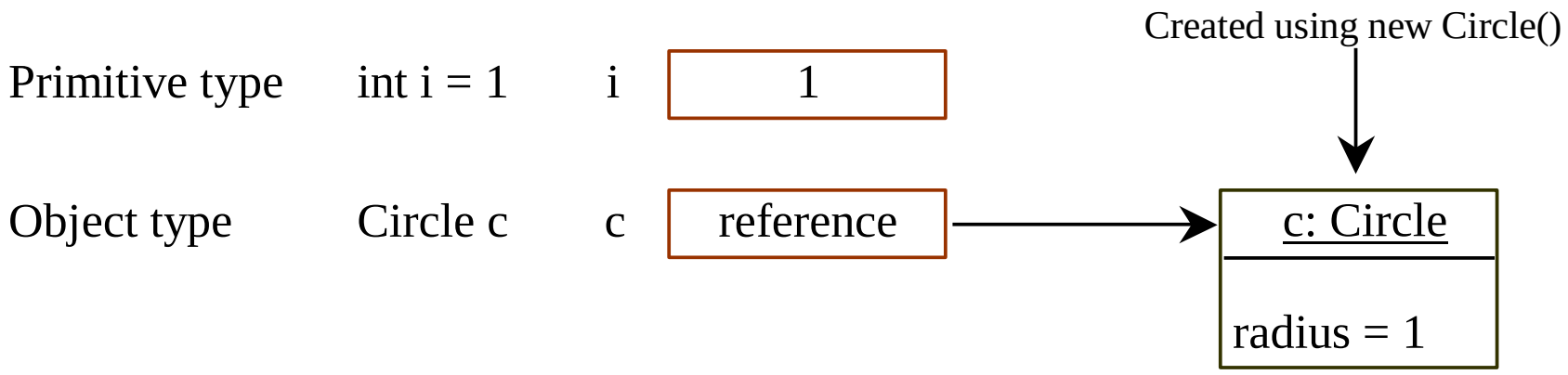
```
public class Test {
  public static void main(String[] args) {
    int x; // x has no default value
    String y; // y has no default value
    System.out.println("x is " + x);
    System.out.println("y is " + y);
  }
}
```



Compile error: variable not initialized



# Differences between Variables of Primitive Data Types and Object Types



# Copying Variables of Primitive Data Types and Object Types

Primitive type assignment i = j

Before:

i 1

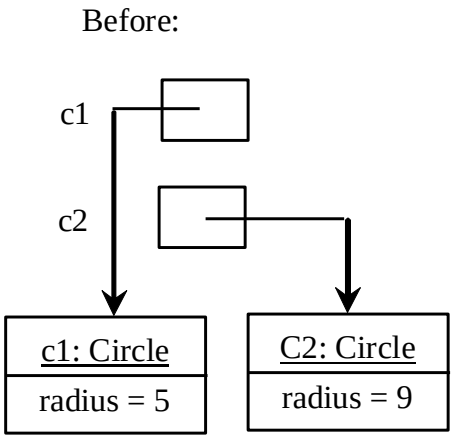
j 2

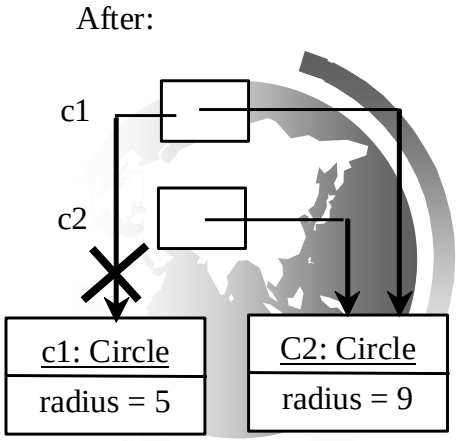
After:

i 2

j 2

Object type assignment c1 = c2



# Garbage Collection

As shown in the previous figure, after the assignment statement c1 = c2, c1 points to the same object referenced by c2. The object previously referenced by c1 is no longer referenced. This object is known as garbage. Garbage is automatically collected by JVM.



# Garbage Collection, cont

TIP: If you know that an object is no longer needed, you can explicitly assign null to a reference variable for the object. The JVM will automatically collect the space if the object is not referenced by any variable.



# Example#1: What is wrong in the following code?

```
class Test {
  public static void main(String[] args) {
        A a = new A();
        a.print();
  }
}
class A {
  String s;
  A(String newS) {
        s = newS;
  }
  public void print() {
        System.out.print(s);
  }
}
```



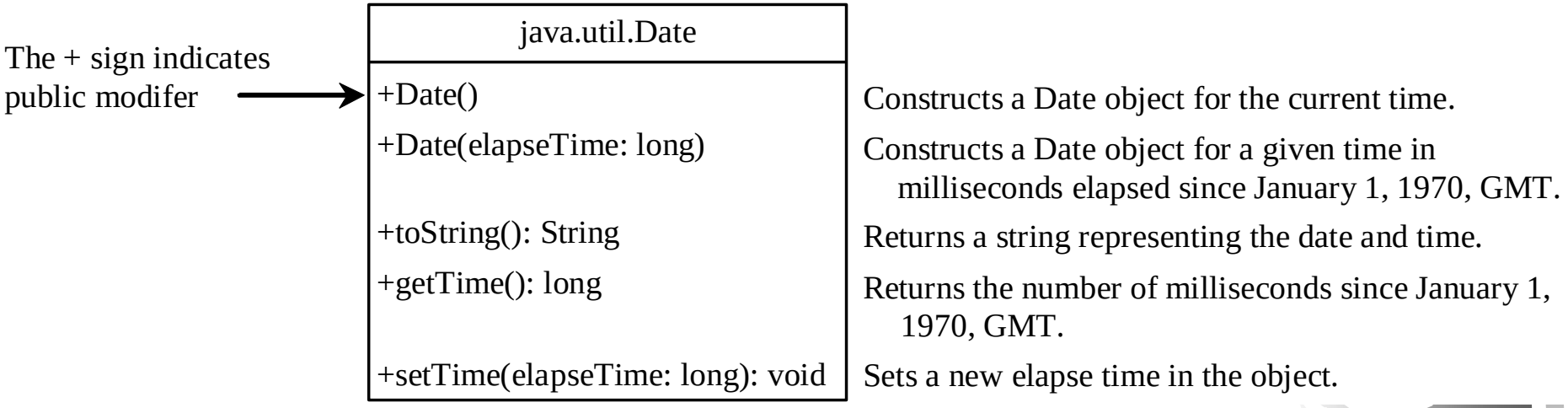
# Example#2: What is the output of the following code?

```
public class A {
  boolean x;
  public static void main(String[] args)
  {
       A a = new A();
       System.out.println(a.x);
  }
}
```



# The Date Class

Java provides a system-independent encapsulation of date and time in the java.util.Date class. You can use the Date class to create an instance for the current date and time and use its toString method to return the date and time as a string.



# The Date Class Example

For example, the following code

```
java.util.Date date = new java.util.Date();
System.out.println(date.toString());
```

displays a string like `Fri Dec 02 10:25:29 EET 2018`



# The Random Class

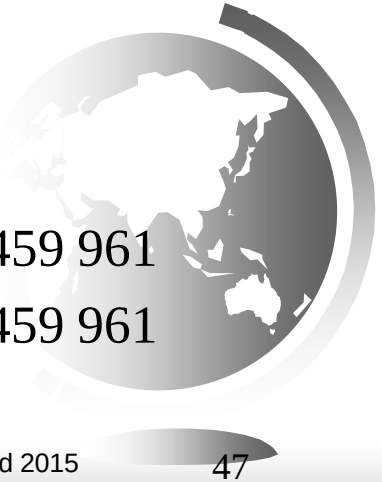
You have used Math.random() to obtain a random double value between 0.0 and 1.0 (excluding 1.0). A more useful random number generator is provided in the java.util.Random class.

| java.util.Random | |
|---|---|
| +Random() | Constructs a Random object with the current time as its seed. |
| +Random(seed: long) | Constructs a Random object with a specified seed. |
| +nextInt(): int | Returns a random int value. |
| +nextInt(n: int): int | Returns a random int value between 0 and n (exclusive). |
| +nextLong(): long | Returns a random long value. |
| +nextDouble(): double | Returns a random double value between 0.0 and 1.0 (exclusive). |
| +nextFloat(): float | Returns a random float value between 0.0F and 1.0F (exclusive). |
| +nextBoolean(): boolean | Returns a random boolean value. |

# The Random Class Example

If two Random objects have the same seed, they will generate identical sequences of numbers. For example, the following code creates two Random objects with the same seed 3.

```
Random random1 = new Random(3);
System.out.print("From random1: ");
for (int i = 0; i < 10; i++)
  System.out.print(random1.nextInt(1000) + " ");
Random random2 = new Random(3);
System.out.print("\nFrom random2: ");
for (int i = 0; i < 10; i++)
  System.out.print(random2.nextInt(1000) + " ");
```



From random1: 734 660 210 581 128 202 549 564 459 961

From random2: 734 660 210 581 128 202 549 564 459 961

# The **Point2D** Class

Java API has a conveninent **Point2D** class in the **javafx.geometry** package for representing a point in a two-dimensional plane.

| javafx.geometry.Point2D | |
|---|---|
| `+Point2D(x: double, y: double)` | Constructs a `Point2D` object with the specified *x*- and *y*-coordinates. |
| `+distance(x: double, y: double): double` | Returns the distance between this point and the specified point $(x, y)$. |
| `+distance(p: Point2D): double` | Returns the distance between this point and the specified point p. |
| `+getX(): double` | Returns the *x*-coordinate from this point. |
| `+getY(): double` | Returns the *y*-coordinate from this point. |
| `+toString(): String` | Returns a string representation for the point. |



Point2D

Run

# Instance Variables, and Methods

Instance variables belong to a specific instance.

Instance methods are invoked by an instance of the class.



# Static Variables, Constants, and Methods

- Static Variables
  - They are shared by all the instances of the class.
  - They store values for the variables in a common memory location.
  - If one object changes the value of a static variable, all objects of the same class are affected.
- Static Methods
  - They are not tied to a specific object.
  - They can be called without creating an instance of the class.



# Static Variables, Constants, and Methods (cont.)

- Constants in a class are shared by all objects of the class. Thus, constants should be declared as **final static**.

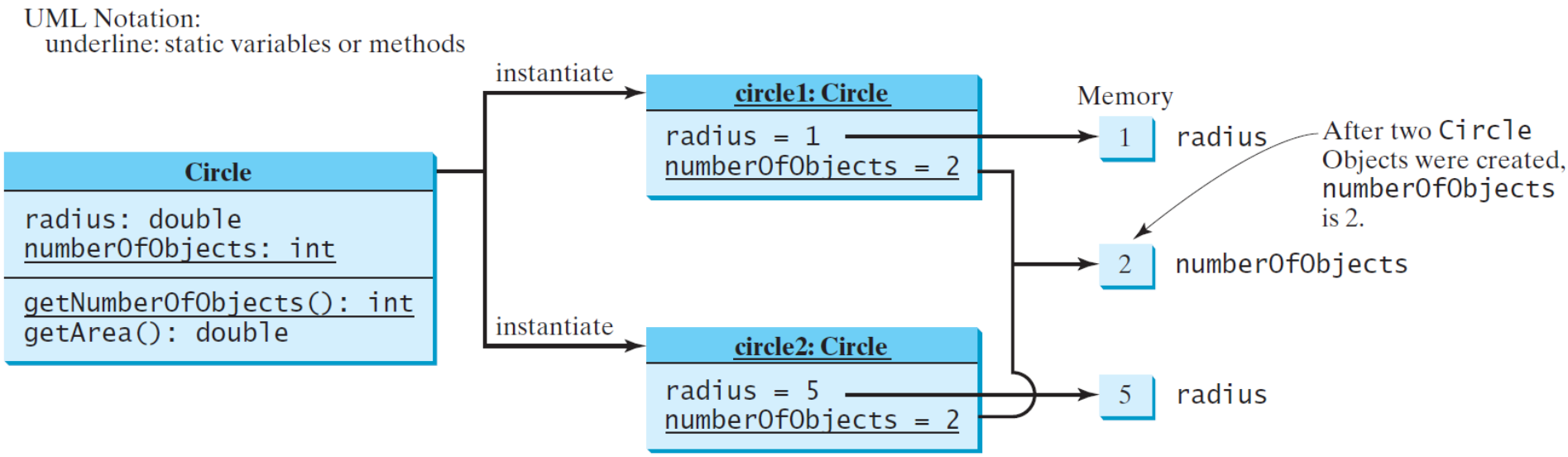
  Example:

```
final static double PI = 3.14159265358979323846;
```

- Static methods and static data fields don't belong to a particular object.
- To declare static variables, constants, and methods, use the **static** modifier.



# Static Variables, Constants, and Methods, cont.



# Example of Using Instance and Class Variables and Method

Objective: Demonstrate the roles of instance and class variables and their uses. This example adds a class variable numberOfObjects to track the number of Circle objects created.



CircleWithStaticMembers



TestCircleWithStaticMembers

Run



# Example of Using Instance and Class Variables and Method (cont.)

- Use **ClassName.methodName(arguments)** to invoke a static method and **ClassName.staticVariable** to access a static variable.

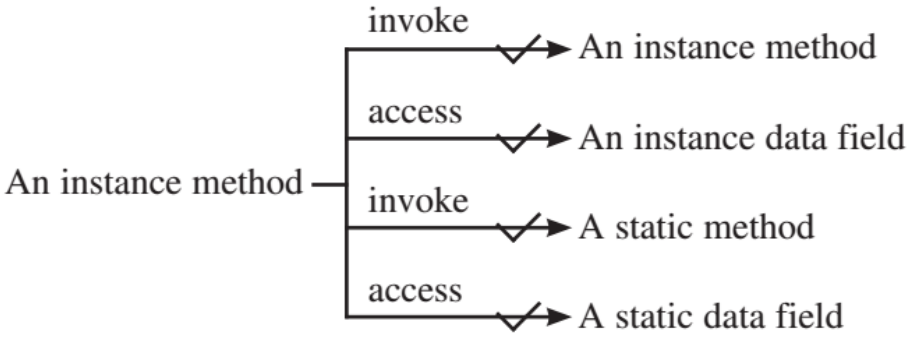
- This improves readability, because this makes the static method and data easy to spot.

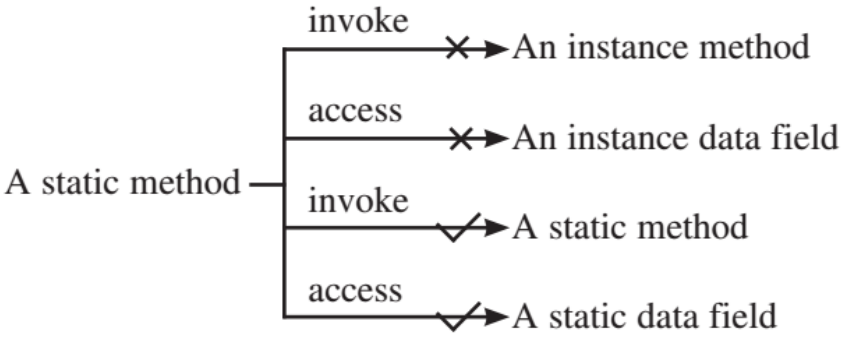
  Example:

  `c1.numberOfObjects` (line 27) and `c2.numberOfObjects` (line 30) are better replaced by `CircleWithStaticMembers.numberOfObjects`.



# Static vs. Instance Methods






# Example #3:

```
1  public class A {
2    int i = 5;
3    static int k = 2;
4
5    public static void main(String[] args) {
6      int j = i; // Wrong because i is an instance variable
7      m1(); // Wrong because m1() is an instance method
8    }
9
10   public void m1() {
11     // Correct since instance and static variables and methods
12     // can be used in an instance method
13     i = i + k + m2(i, k);
14   }
15
16   public static int m2(int i, int j) {
17     return (int)(Math.pow(i, j));
18   }
19 }
```



# Example #3 (cont.)

```
1  public class A {
2    int i = 5;
3    static int k = 2;
4
5    public static void main(String[] args) {
6      A a = new A();
7      int j = a.i; // OK, a.i accesses the object's instance variable
8      a.m1(); // OK. a.m1() invokes the object's instance method
9    }
10
11   public void m1() {
12     i = i + k + m2(i, k);
13   }
14
15   public static int m2(int i, int j) {
16     return (int)(Math.pow(i, j));
17   }
18 }
```

# Common Design Error

- It is a common design error to define an instance method that should have been defined as static. For example, the method **factorial(int n)** should be defined as static, as shown next, because it is independent of any specific instance.

```
public class Test {
  public int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i ++)
      result *= i;

    return result;
  }
}
```

(a) Wrong design

```
public class Test {
  public static int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++)
      result *= i;

    return result;
  }
}
```

(b) Correct design

# Visibility Modifiers and Accessor/Mutator Methods

By default, the class, variable, or method can be accessed by any class in the same package.

- `public`

  The class, data, or method is visible to any class in any package.

- `private`

  The data or methods can be accessed only by the declaring class.

The get and set methods are used to read and modify private properties.



```java
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```java
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```java
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

The *private* modifier restricts access to within a class.
The *default* modifier restricts access to within a package.
The *public* modifier enables unrestricted access.



```java
package p1;

public class C1 {
  public int x;
  int y;
  private int z;

  public void m1() {
  }
  void m2() {
  }
  private void m3() {
  }
}
```

```java
package p1;

public class C2 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    can access o.y;
    cannot access o.z;

    can invoke o.m1();
    can invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```java
package p2;

public class C3 {
  void aMethod() {
    C1 o = new C1();
    can access o.x;
    cannot access o.y;
    cannot access o.z;

    can invoke o.m1();
    cannot invoke o.m2();
    cannot invoke o.m3();
  }
}
```

```java
package p1;

class C1 {
  ...
}
```

```java
package p1;

public class C2 {
  can access C1
}
```

```java
package p2;

public class C3 {
  cannot access C1;
  can access C2;
}
```

The *private* modifier restricts access to within a class.
The *default* modifier restricts access to within a package.
The *public* modifier enables unrestricted access.

# NOTE

An object cannot access its private members, as shown in (b). It is OK, however, if the object is declared in its own class, as shown in (a).

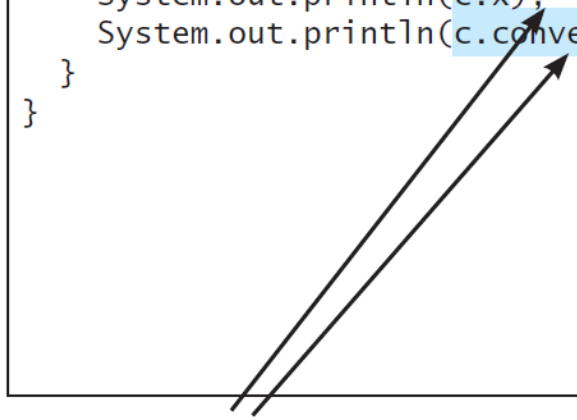
```
public class C {
  private boolean x;

  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }

  private int convert() {
    return x ? 1 : -1;
  }
}
```

(a) This is okay because object `c` is used inside the class `C`.

```
public class Test {
  public static void main(String[] args) {
    C c = new C();
    System.out.println(c.x);
    System.out.println(c.convert());
  }
}
```



(b) This is wrong because `x` and `convert` are private in class `C`.

# Why Data Fields Should Be private?

To protect data.

To make code easy to maintain.



# Data Field Encapsulation

- To prevent direct modifications of data fields, you should declare the data fields private, using the **private** modifier.
- This is known as *data field encapsulation*



# Data Field Encapsulation (cont.)

- A private data field cannot be accessed by an object from outside the class that defines the private field.
- However, a client often needs to retrieve and modify a data field.
- To make a private data field accessible, provide a *getter* method to return its value.
- To enable a private data field to be updated, provide a *setter* method to set a new value.



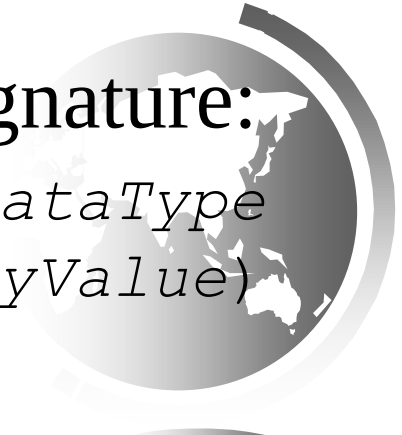
# Data Field Encapsulation (cont.)

- A getter method is also referred to as an *accessor* and a setter to a *mutator*.
- A getter method has the following signature:

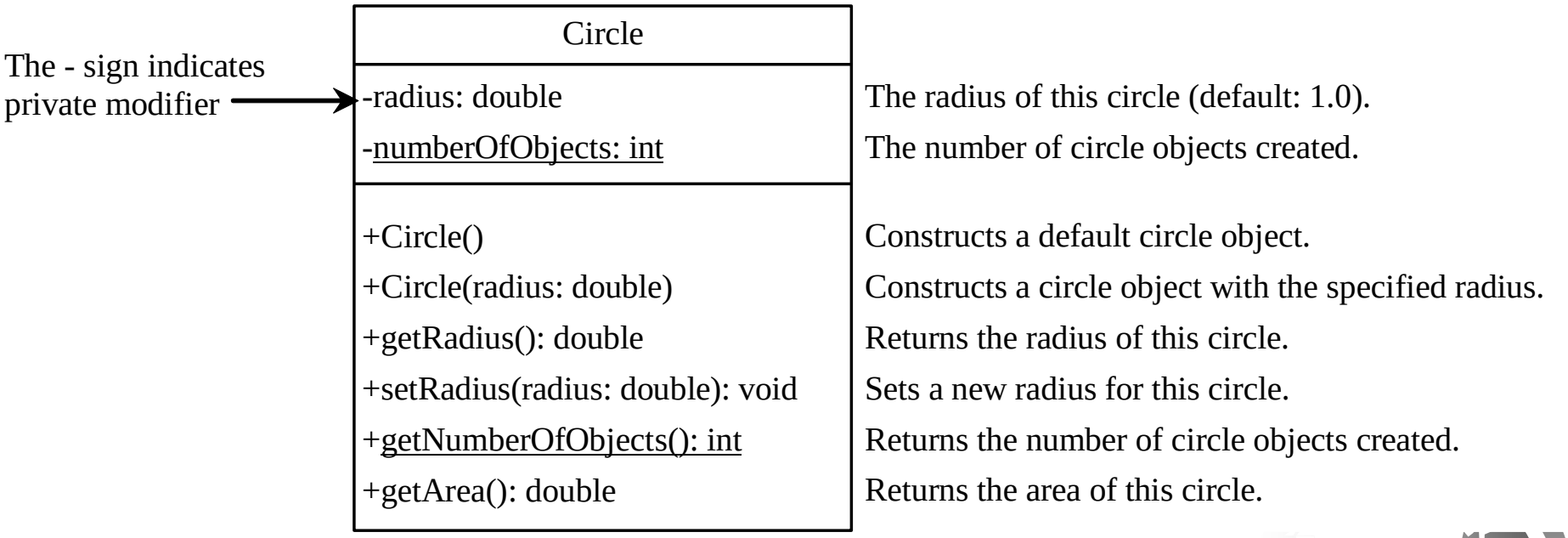
```
public returnType getPropertyName()
```

- A setter method has the following signature:

```
public void setPropertyName(dataType
                            propertyValue)
```



# Example of Data Field Encapsulation




CircleWithPrivateDataFields

TestCircleWithPrivateDataFields

Run

# Passing Objects to Methods

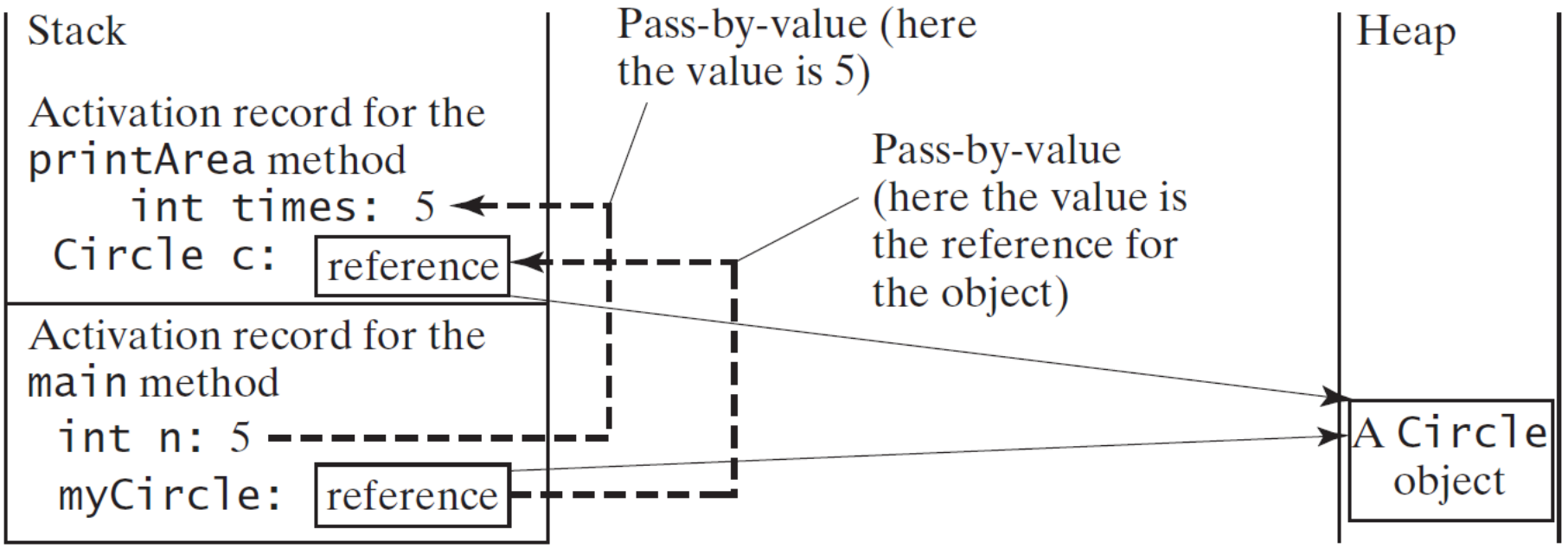
- Passing by value for primitive type value (the value is passed to the parameter)
- Passing by value for reference type value (the value is the reference to the object)
- Any changes to the object that occur inside the method body will affect the original object that was passed as the argument



TestPassObject

Run

# Passing Objects to Methods, cont.



# Array of Objects

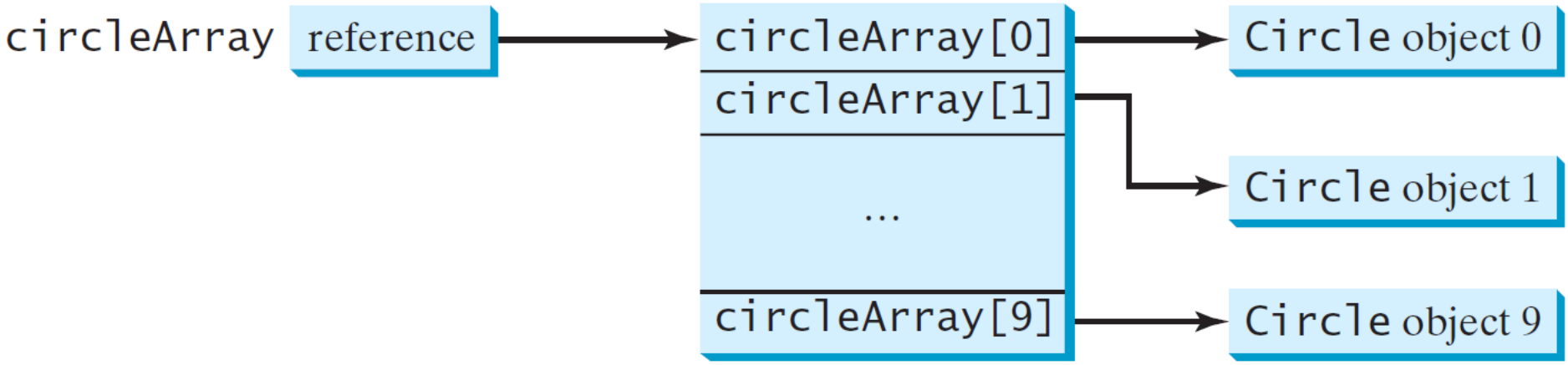
```
Circle[] circleArray = new Circle[10];
```

- An array of objects is actually an *array of reference variables*.
- So invoking circleArray[1].getArea() involves two levels of referencing.
- circleArray references to the entire array.
- circleArray[1] references to a Circle object.



# Array of Objects, cont.

```
Circle[] circleArray = new Circle[10];
```






# Array of Objects, cont.

## Summarizing the areas of the circles



TotalArea



Run

# Immutable Objects and Classes

- If the contents of an object cannot be changed once the object is created, the object is called an *immutable object* and its class is called an *immutable class*.

- If you delete the set method in the Circle class, the class would be immutable because radius is private and cannot be changed without a set method.



# Immutable Objects and Classes (cont.)

- A class with all private data fields and without mutators is not necessarily immutable.

- For example, the following class Student has all private data fields and no mutators, but it is mutable.



# Example

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }

  public void setYear(int newYear) {
    year = newYear;
  }
}
```

```
public class Test {
  public static void main(String[] args) {
    Student student = new Student(111223333, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010); // Now the student birth year is changed!
  }
}
```



# What Class is Immutable?

- For a class to be immutable,
  - it must mark all data fields *private* and
  - provide no *mutator (setter) methods* and
  - no accessor methods that would return a *reference to a mutable data field object*.



# Scope of Variables

- The scope of instance and static variables is the entire class.
- They can be declared anywhere inside a class.
- The scope of a local variable starts from its declaration and continues to the end of the block that contains the variable.
- A local variable must be initialized explicitly before it can be used.



# Scope of Variables (cont.)

- If a local variable has the same name as a class's variable, the local variable takes precedence and the class's variable with the same name is *hidden*.

```
public class F {
  private int x = 0; // Instance variable
  private int y = 0;

  public F() {
  }

  public void p() {
    int x = 1; // Local variable
    System.out.println("x = " + x);
    System.out.println("y = " + y);
  }
}
```



# The this Keyword

- The this keyword is the name of a reference that refers to an object itself.
- One common use of the this keyword is reference a class's *hidden data fields*.
- Another common use of the this keyword to enable a constructor to invoke another constructor of the same class.
- this must be first statement in constructor



# Reference the Hidden Data Fields

```
public class F {
  private int i = 5;
  private static double k = 0;

  void setI(int i) {
    this.i = i;
  }

  static void setK(double k) {
    F.k = k;
  }
}
```

```
Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
   this.i = 10, where this refers f1

Invoking f2.setI(45) is to execute
   this.i = 45, where this refers f2
```



# Calling Overloaded Constructor

```
public class Circle {
  private double radius;

  public Circle(double radius) {
    this.radius = radius;
  }

  public Circle() {
    this(1.0);
  }

  public double getArea() {
    return this.radius * this.radius * Math.PI;
  }
}
```

this must be explicitly used to reference the data field radius of the object being constructed

this is used to invoke another constructor

this must appear first in the constructor before any other statements

Every instance variable belongs to an instance represented by this, which is normally omitted

# Example #4: What is wrong with each of the following programs

```
 1  public class ShowErrors {
 2    public void method1() {
 3      Circle c;
 4      System.out.println("What is radius "
 5        + c.getRadius());
 6      c = new Circle();
 7    }
 8  }
```

```
 1  public class ShowErrors {
 2    public static void main(String[] args) {
 3      C c = new C(5.0);
 4      System.out.println(c.value);
 5    }
 6  }
 7
 8  class C {
 9    int value = 2;
10  }
```



# Example #5:

Suppose that the class **F** is defined in (a). Let **f** be an instance of **F**. Which of the statements in (b) are correct?

```
public class F {
  int i;
  static String s;

  void imethod() {
  }

  static void smethod() {
  }
}
```

(a)

```
System.out.println(f.i);
System.out.println(f.s);
f.imethod();
f.smethod();
System.out.println(F.i);
System.out.println(F.s);
F.imethod();
F.smethod();
```

(b)



# Example #6:

In the following code, **radius** is private in the **Circle** class, and **myCircle** is an object of the **Circle** class. Does the highlighted code cause any problems? If so, explain why.

```
public class Circle {
  private double radius = 1;

  /** Find the area of this circle */
  public double getArea() {
    return radius * radius * Math.PI;
  }

  public static void main(String[] args) {
    Circle myCircle = new Circle();
    System.out.println("Radius is " + myCircle.radius);
  }
}
```



# Example #7: Show the output of the following code:

```
public class Test {
  public static void main(String[] args) {
    T t1 = new T();
    T t2 = new T();
    System.out.println("t1's i = " +
      t1.i + " and j = " + t1.j);
    System.out.println("t2's i = " +
      t2.i + " and j = " + t2.j);
  }
}

class T {
  static int i = 0;
  int j = 0;

  T() {
    i++;
    j = 1;
  }
}
```



# Example #8:What is wrong in the following code?

```
1  public class C {
2    private int p;
3
4    public C() {
5      System.out.println("C's no-arg constructor invoked");
6      this(0);
7    }
8
9    public C(int p) {
10     p = p;
11   }
12
13   public void setP(int p) {
14     p = p;
15   }
16 }
```

